



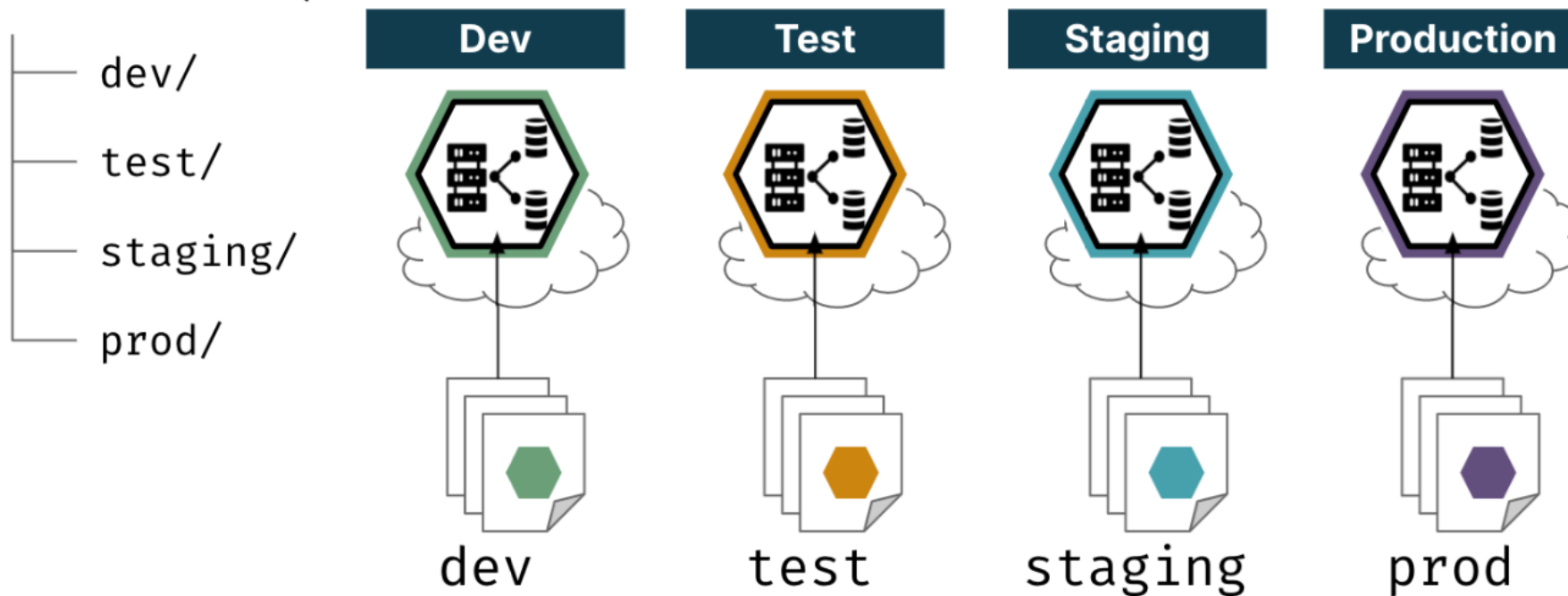
Infrastructure as Code

Iac (Infrastructure as Code) Summary

◎ Iac (Infrastructure as Code)

- ▶ IaC란 코드 작성 후 실행으로 인프라를 생성, 배포, 수정, 정리하는 것을 의미한다.
- ▶ IaC는 하드웨어 관리 및 운영 관리에 관한 모든 작업을 코드 형태의 관리를 수행하는 것을 목적으로 한다.
- ▶ IaC는 서버, 네트워크, DB, APP, 자동화된 테스트 및 배포 등의 기능을 구현하며 DevOps의 핵심요소가 된다.
- ▶ 코드형 인프라를 구현하는 도구의 종류는 다양하며, 대표적으로 테라폼, 앤서블, 쿠버네티스 등이 있다.

infrastructure/



Iac (Infrastructure as Code) Summary

◎ IaC (Infrastructure as Code) 구현도구 5가지 범주

▣ 애드혹(ad-hoc) 스크립트

- ▶ 수행 될 작업을 단계별로 나누어 Bash, Python과 같은 스크립트 언어를 사용하여 각 단계를 코드로 정의 후 실행한다.
- ▶ 애드혹 스크립트는 단순한 형태의 작업을 자동화 하는데 적합하며, 모든 스크립트는 각 서버에서 수동실행 된다.

※ 범용 언어를 사용하는 애드혹 스크립트의 단점

- 다양한 형태의 서버, 네트워크, DB 등을 관리하는 대규모 인프라 환경에 적용하여 사용하기 어렵다.
- 범용 프로그래밍 언어의 사용. (스크립트 작성 형식이 정해져 있지 않다.)

▣ 구성 관리 도구

- ▶ 관리 대상 서버에 소프트웨어를 설치하거나 운영하는데 특화되어있는 코드형 인프라 전용 도구 (앤서블, 셰프, 퍼핏, 솔트스택 등)
- ▶ 구성 관리 도구의 경우 전용 API 및 코드 작성 형식이 대체로 정해져 있어 코드 탐색이 용이하다. (코딩 규칙)
- ▶ 멱등성(어떠한 작업을 여러 번 실행하여도 결과는 항상 같아야 한다)을 제공한다.
- ▶ 분산형 구조를 지원한다. (원격으로 수 많은 서버를 동시에 관리 할 수 있다.)

Iac (Infrastructure as Code) Summary

▣ 서버 템플릿 도구

- ▶ 구성 관리 도구의 대안으로 자주사용되며 대표적인 서버 템플릿 도구로는 도커, 패커, 베이그런트 등이 있다. ([이미지 배포 도구](#))
- ▶ 배포 이미지에는 운영체제, 소프트웨어, 파일 등 서버 운영에 필요한 모든 구성요소를 이미지 내부에 포함하고 있다.
- ▶ 서버 템플릿 도구는 [불변 인프라\(Immutable infrastructure\)](#)를 구현하는 핵심요소로 사용된다.
- ▶ 불변 인프라는 서버의 변경사항을 기존 서버에 덮어쓰우는 것이 아닌 새로운 이미지를 만들어 새로운 서버를 배포하는 개념이 된다.
- ▶ 불변 인프라를 구현 할 경우 새롭게 배포된 부분을 관리하기가 수월해지며, 기존 서버 시스템의 변경사항 이력 관리가 필요 없게 된다.
- ▶ 불변 인프라의 단점으로는 새로운 서버를 매번 배포해야 하므로 배포 시간이 오래 걸린다는 단점이 존재한다. (컨테이너 가상화 x)

※ 서버 템플릿 도구는 각 도구들마다 사용 목적에 차이가 있다.

- 도커(개별 응용 프로그램 이미지), 패커(AWS AMI), 베이그런트(개발 컴퓨터에서 실행되는 버추얼박스 이미지)

▣ 오케스트레이션 도구

- ▶ 서버 템플릿 도구에서 만들어진 이미지를 적절한 배포전략에 맞추어 배포하거나 배포된 서버를 관리하는 전용도구
- ▶ 대표적인 도구로는 도커 컨테이너를 효과적으로 관리 할 수 있는 [쿠버네티스 오케스트레이션 도구](#)가 있다.
- ▶ 그 외 각종 오케스트레이션 도구도 존재하며, Public Cloud 공급업체에서 전용 서비스로 지원하는 도구들도 존재한다. (AWS ECS)

▣ 프로비저닝 도구

- ▶ 서버 템플릿 도구, 구성 관리 도구가 각 서버에서 실행되는 코드를 정의한다면, 프로비저닝은 [서버 자체를 생성하는 도구](#)로 사용된다.
- ▶ 프로비저닝 도구는 서버만 생성하는 것이 아닌 인프라에 관한 대부분의 구성 요소를 프로비저닝 할 수 있다. (네트워크, LB, DB)
- ▶ 대표적으로 테라폼, AWS CloudFormation, 오픈스택 Heat 등의 도구가 있다.

Iac (Infrastructure as Code) Summary

◎ IaC (Infrastructure as Code)의 장점

■ 자급식 배포 (Self-Service)

- ▶ 인프라 배포작업은 배포 작업을 수행하는데 필요한 작업 프로세스를 알고 있는 소수의 시스템 관리자만 배포를 수행 할 수 있다.
- ▶ 인프라를 코드로 정의 할 경우 전체 배포 프로세스를 자동화 할 수 있으며 개발자는 필요할 때마다 자체적인 배포가 가능해진다.

■ 속도와 안정성 (Speed And Safety)

- ▶ 배포 프로세스를 자동화하면 사람이 진행하는 것보다 훨씬 빠른 속도로 배포를 진행 할 수 있다.
- ▶ 자동화 된 배포 프로세스는 일관되고 반복 가능하며, 수동 배포 때보다 오류가 적게 발생하기 때문에 안전한 배포가 가능해진다.

■ 문서화 (Documentation)

- ▶ 시스템 관리자만 인프라에 관한 정보를 독점하는 것이 아닌, 누구나 읽을 수 있는 소스 파일 형태로 인프라 상태를 나타낼 수 있다.
- ▶ 즉, 코드형 인프라는 문서의 역할을 하며, 조직의 모든 사람이 인프라 구조를 이해하고 업무를 수행 할 수 있도록 해준다.

■ 버전 관리 (Version Control)

- ▶ 인프라의 변경 사항이 소스 파일에 기록되어 있으므로, 인프라 변경사항 버전관리를 쉽게 할 수 있다.
- ▶ 또한, 소스 파일에 인프라 변경 내역이 남아 있으므로 시스템에 문제가 생겼을 경우 문제 발생지점을 찾기 수월하다.
- ▶ 만약 문제 발생지점을 찾지 못한경우에는 이전 인프라로 쉽게 되돌아 갈 수 있어 디버깅을 돕는 강력한 도구의 역할 수행하게 된다.

■ 유효성 검증 (Validation) 및 재사용성 (Reuse)

- ▶ 인프라 상태가 코드로 정의되어 있어, 코드가 변경 될 때마다 검증을 수행하고 일련의 자동화 된 테스트를 실행 할 수 있다.
- ▶ 인프라를 재사용 가능한 모듈로 패키징 할 수 있으므로, 문서화되고 검증 된 모듈로 일관 된 배포를 할 수 있다.



HashiCorp Terraform

Iac (Infrastructure as Code) : Terraform

◎ Terraform

- ▶ 테라폼은 해시코프사가 Go언어로 개발한 오픈소스 도구로 여러 클라우드 공급자를 지원한다.
- ▶ 테라폼은 각 클라우드 플랫폼의 API를 호출하여 리소스를 생성 할 수 있으며, 해당 API 호출을 코드로 정의한다.
- ▶ 테라폼은 마스터 서버나 에이전트 설치가 필요 없으므로 구성관리가 간편하고 추가 서버를 구성 할 필요가 없다.
- ▶ 테라폼은 "선언적 언어"를 지향하며, "선언적 언어"란 구현하려는 최종 상태를 지정하는 코드를 의미한다.
- ▶ 선언적 언어를 사용하게 되면, 인프라의 최종 상태를 선언하는데 집중하여 코드를 작성 할 수 있다.
- ▶ Terraform DOCS URL : <https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources>
- ▶ AWS Providers DOCS URL : <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- ▶ Download URL : https://releases.hashicorp.com/terraform/1.4.6/terraform_1.4.6_windows_amd64.zip

※ 절차적 언어

- 절차적 언어는 인프라의 마지막 상태 정보를 기록하지 않는다. (마지막으로 배포 된 인프라의 상태를 기억하지 않는다.)
- 절차적 언어는 재사용 가능성을 제한 할 수 있다. (규모가 크고 복잡한 인프라에서는 실시간으로 인프라의 변화가 발생한다.)

EX:) 기존 10대의 서버가 배포 된 상황에서 트래픽 증가로 인해 추가로 5대의 서버를 배포해야하는 상황

▶ [절차적 언어]

- 기존 코드에서 서버의 수를 정의하는 영역에서 서버의 수를 "10" -> "15"로 변경 후 코드를 실행
- 기존 10대의 서버가 배포 된 상태에서 추가로 15대의 서버가 배포 되므로 총 25대의 서버를 유지한다.

▶ [선언적 언어]

- 기존 코드에서 서버의 수를 정의하는 영역에서 서버의 수를 "10" -> "15"로 변경 후 코드를 실행
- 기존 10대의 서버가 배포 된 상태에서 추가로 5대의 서버만 배포하므로 총 15대의 서버를 유지한다.

IaC (Infrastructure as Code) : Terraform

◎ Terraform의 장단점

▣ 장점

- ▶ 별도의 콘솔 로그인 없이 인프라 리소스를 관리하고 운영 할 수 있다.
- ▶ 항상 일관성 있는 배포를 수행 할 수 있으며, 자동화 된 관리를 지원하여 빠른 배포를 수행 할 수 있다.
- ▶ 인프라 배포 전 유효성 검증을 통해 배포 작업 시 발생하는 문제를 예방 할 수 있다.

▣ 단점

- ▶ 이미 배포되어있는 대규모 인프라를 한번에 테라폼 코드로 마이그레이션하기 어렵다.
- ▶ 모듈형식의 테라폼 코드를 작성하지 않을 경우 배포작업 마다 전체 인프라를 지우고 새로 배포하는 형식을 취하게 된다. (배포속도 저하)
- ▶ 테라폼이 지원하는 유효성 검사 작업은 100% 신뢰 할 수 없다. (Plan 성공 -> Apply 실패)
- ▶ 별도의 HCL문법을 익혀야 하며(러닝 커브), 인프라 배포 전략이 제한적일 수 있다.

```
resource "aws_instance" "web" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"

  tags = {
    Name = "HelloWorld"
  }
}
```

```
resource "aws_subnet" "my_subnet" {
  vpc_id          = aws_vpc.my_vpc.id
  cidr_block      = "172.16.10.0/24"
  availability_zone = "us-west-2a"

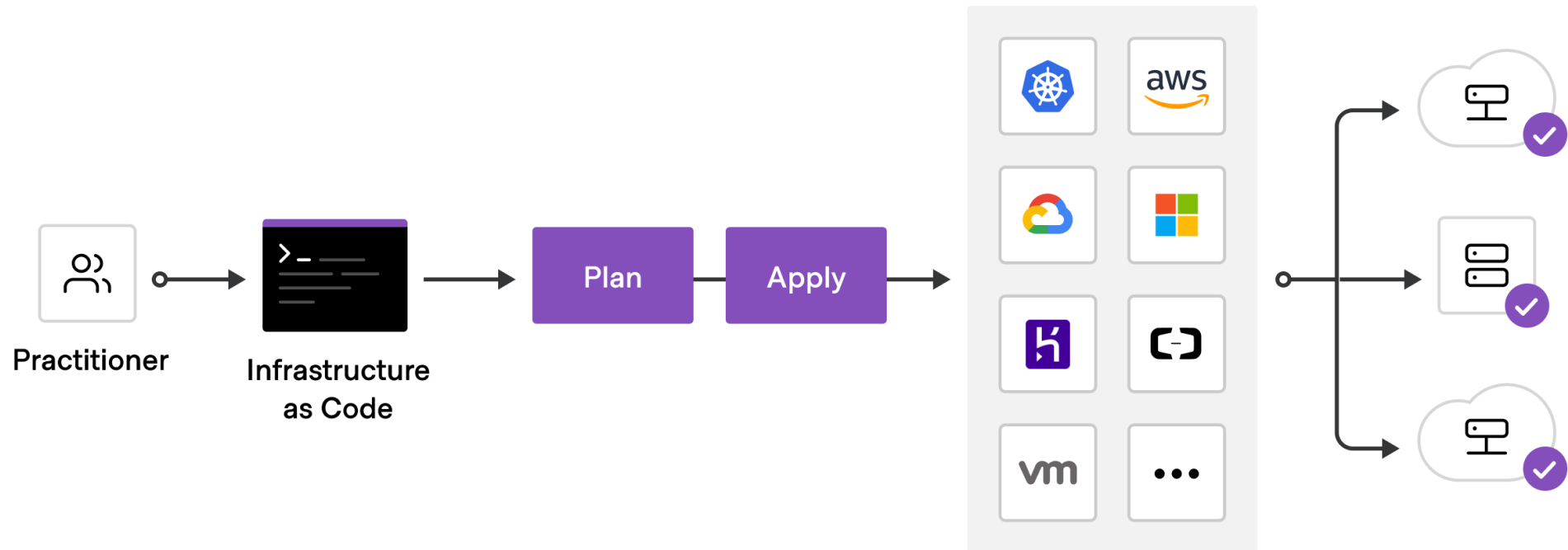
  tags = {
    Name = "tf-example"
  }
}
```

IaC (Infrastructure as Code) : Terraform

◎ Terraform Workflow

- ▶ 1. 테라폼 코드를 새롭게 작성하거나 기존 테라폼 코드를 수정 후 배포작업을 시작
- ▶ 2. 현재 구성되어있는 인프라의 상태를 확인 ([Terraform State](#))
- ▶ 3. 현재 인프라 상태와 테라폼 코드에서 요구하는 상태를 비교 후 변경사항 체크 후 화면에 출력 ([Plan](#))
- ▶ 4. 변경사항 체크 후 실제 인프라 변경 작업을 수행 ([apply](#))
- ▶ 5. 테라폼 코드로 배포 된 인프라 리소스 삭제 ([destory](#))

※ 테라폼은 리소스간 의존 관계를 별도로 명시하지 않아도, 테라폼 스스로 리소스간 의존 관계를 파악하여 병렬 배포 작업을 수행한다.



IaC (Infrastructure as Code) : Terraform

◎ Terraform 기본설정 및 AWS Provider 정의

▣ <작업경로> : C:\Terraform\main.tf

- ▶ 테라폼 코드를 저장하는 소스파일의 확장자는 ".tf"로 정의한다.
- ▶ "main.tf" : Terraform 설정 및 테라폼에서 사용 할 Provider 지정
- ▶ HCL Attribute 구문 : "식별자(Argument Name) = 값(Value)"
- ▶ Attribute Block 정의 : "{ }"

※ TIP : HCL 문법 정리 명령어 : "terraform fmt"

[main.tf 구성요소]

- ▶ terraform {...} : 테라폼 설정 정의 영역
- ▶ provider {...} : 테라폼에서 사용 할 Provider 지정 (AWS)
 - region : AWS Resource를 생성 할 Region을 정의
 - profile : AWS CLI 인증정보를 담고있는 Profile을 정의
- ▶ required_providers {...} : 지정 된 Provider 설정 정의 영역
 - source : Provider Plugin 다운로드 경로
 - version : Provider Plugin 버전 명시

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "ap-northeast-2"  
  profile = "terraform_user"  
}
```

IaC (Infrastructure as Code) : Terraform

◎ Terraform 초기구성 작업

▣ [CMD] : **C:\Terraform>terraform init**

Initializing the backend...

Initializing provider plugins...

- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.10.0...
- Installed hashicorp/aws v5.10.0 (signed by HashiCorp)

Terraform has created a lock file `.terraform.lock.hcl` to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been **successfully** initialized!

- ▶ Provider Resource, Data Source 다운로드 및 Terraform 초기구성 작업
- ▶ 테라폼 및 Provider 설정이 변경되었을 경우 반드시 초기구성 작업을 다시 수행해야한다.
- ▶ 초기구성 작업 완료 후 생성 파일 : ".terraform.lock.hcl"
- ▶ ".terraform.lock.hcl" 잠금 파일 (Team 단위 프로젝트에서 Resource 동시접근을 방지)



Resource & Data Source

Iac (Infrastructure as Code) : Terraform

◎ Terraform AWS EC2 Resource 생성

▣ <작업경로> : C:\Terraform\main.tf

▶ Resource : VPC, EC2 Instance등의 Resource를 정의

▶ EC2 Resource Docs : <https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance>

```
resource "<PROVIDER>_<TYPE>" "<NAME>" {  
  [ CONFIG ... ]  
}
```

- <PROVIDER> : 공급자 이름
- <TYPE> : 공급자의 리소스의 유형 (EC2, VPC 등)
- <NAME> : 다른 테라폼 코드에서 해당 리소스를 참조하기 위한 식별 이름
- [CONFIG] : 특정 리소스를 생성하기위한 설정 값 (AMI, INSTANCE 유형 등)

```
resource "aws_instance" "Example" {  
  ami           = "ami-0ea4d4b8dc1e46212"  
  instance_type = "t2.micro"  
}
```

Iac (Infrastructure as Code) : Terraform

◎ Terraform AWS EC2 Resource 생성 (EX.1)

▣ [CMD] : **C:\Terraform>terraform plan**

- ▶ terraform plan 명령은 해당 테라폼 코드가 동작했을때 변경사항을 기호로 출력
- ▶ 추가 : ("+") / 삭제 : ("-") / 변경 : ("~") (terraform 코드를 실행 전 변경사항을 체크)

Terraform used the selected providers to generate the following execution plan.

Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

aws_instance.Example will be created

```
+ resource "aws_instance" "Example" {  
    + ami                                = "ami-0ea4d4b8dc1e46212"  
    ... 생략 ...  
    + instance_type                     = "t2.micro"  
    ... 생략 ...
```

Terraform will perform the following actions:

Plan: 1 to add, 0 to change, 0 to destroy.

Iac (Infrastructure as Code) : Terraform

◎ Terraform AWS EC2 Resource 생성 (EX.1)

▣ [CMD] : **C:\Terraform>terraform apply**

- ▶ terraform apply 명령은 테라폼 코드를 이용하여 실제 리소스를 정의한다. (Plan + 실제 리소스 정의 작업을 동시 수행)
- ▶ AWS 콘솔 EC2 메뉴에서 새로운 Instance가 생성되었는지 확인

[... Plan 수행 ...]

Terraform will perform the following actions:

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: **yes**

aws_instance.Example: Creating...

aws_instance.Example: Still creating... [10s elapsed]

aws_instance.Example: Creation complete after 32s [id=i-0814ae8ce9bc3ce67]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

IaC (Infrastructure as Code) : Terraform

◎ Terraform AWS EC2 Resource 수정 및 삭제 (EX.2)

▣ <작업경로> : C:\Terraform\main.tf

- ▶ 기존 AWS EC2 Resource 코드를 수정 후 재 배포작업을 수행 (Instance Tag 지정)
- ▶ AWS EC2 Docs : <https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance>

```
provider "aws" {  
  region = "ap-northeast-2"  
  profile = "terraform_user"  
}  
  
resource "aws_instance" "Example" {  
  ami           = "ami-0ea4d4b8dc1e46212"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "Terraform_EC2_Example"  
  }  
}
```

Iac (Infrastructure as Code) : Terraform

© Terraform AWS EC2 Resource 수정 및 삭제 (EX.2)

▣ [CMD] : **C:\Terraform>terraform apply**

▶ AWS 콘솔 EC2 메뉴에서 기존 Instance에 Tag가 지정되었는지 확인

[... Plan 수행 ...]

Terraform will perform the following actions:

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: **yes**

aws_instance.Example: Modifying... [id=i-0814ae8ce9bc3ce67]

aws_instance.Example: Modifications complete after 0s [id=i-0814ae8ce9bc3ce67]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

IaC (Infrastructure as Code) : Terraform

◎ Terraform AWS EC2 Resource 수정 및 삭제 (EX.2)

▣ EC2 Instance 변경사항 확인

[수정 전]

인스턴스 (1/1) 정보										
Q 인스턴스를 속성 또는 (case-sensitive) 태그로 찾기										
인스턴스 상태 = running X 필터 지우기										
☒	Name ▼	인스턴스 ID	인스턴스 상태 ▼	인스턴스 유형 ▼	상태 검사	경보 상태	가용 영역 ▼	퍼블릭 IPv4 DNS ▼	퍼블릭 IPv4 ... ▼	탄력적 IP
☒	-	i-0814ae8ce9bc3ce67	실행 중	t2.micro	2/2개 검사 통과...	경보 없음 +	ap-northeast-2c	ec2-52-79-228-10.ap-n...	52.79.228.10	-

[수정 후]

인스턴스 (1) 정보										
Q 인스턴스를 속성 또는 (case-sensitive) 태그로 찾기										
인스턴스 상태 = running X 필터 지우기										
☐	Name ▼	인스턴스 ID	인스턴스 상태 ▼	인스턴스 유형 ▼	상태 검사	경보 상태	가용 영역 ▼	퍼블릭 IPv4 DNS ▼	퍼블릭 IPv4 ... ▼	탄력적 IP
☐	Terraform_EC...	i-0814ae8ce9bc3ce67	실행 중	t2.micro	2/2개 검사 통과...	경보 없음 +	ap-northeast-2c	ec2-52-79-228-10.ap-n...	52.79.228.10	-

IaC (Infrastructure as Code) : Terraform

© Terraform AWS EC2 Resource 수정 및 삭제 (EX.2)

▣ [CMD] : **C:\Terraform>terraform destroy**

▶ AWS 콘솔 EC2 메뉴에서 Instance가 삭제되었는지 확인

[... Plan 수행 ...]

Terraform will perform the following actions:

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.

There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: **yes**

aws_instance.Example: Destroying... [id=i-0814ae8ce9bc3ce67]

aws_instance.Example: Still destroying... [id=i-0814ae8ce9bc3ce67, 10s elapsed]

aws_instance.Example: Destruction complete after 40s

Destroy complete! Resources: 1 destroyed.

Iac (Infrastructure as Code) : Terraform

◎ Terraform Resource 참조

- ▶ Resource 참조: 테라폼 코드로 정의 된 다양한 Resource간 연결을 지원
- ▶ 테라폼은 각 Resource의 종속성을 자동으로 파악 후 Resource를 생성한다.
- ▶ Resource 참조 및 종속성 자동파악 (EX) : (보안그룹 생성 -> EC2 인스턴스 생성 및 보안그룹 연결)

[테라폼 코드내에서 정의 된 Resource간 참조 구문]

`<PROVIDER>_<TYPE>.<NAME>.<ATTRIBUTE>`

- `<PROVIDER>` : 공급자 이름
 - `<TYPE>` : 공급자의 리소스의 유형 (`EC2`, `Security_Group` 등)
 - `<NAME>` : 다른 테라폼 코드에서 해당 리소스를 참조하기 위한 식별 이름
 - `<ATTRIBUTE>` : `NAME`에서 지정 된 리소스가 내보낸 속성
- `>. Security Group ID ATTRIBUTE ( id = Security Group 식별값 )`

IaC (Infrastructure as Code) : Terraform

◎ Terraform Resource 참조 (EX.3)

- ▶ 기존 EC2 Instance, Security Group Resource 정의영역 **주석 후 작업**
- ▶ Security-Group Docs : https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/security_group
- ▶ EC2 Instance에서는 여러 개의 Security-Group을 참조 할 수 있으므로, List Type으로 참조를 정의한다.

```
# SSH 원격접속을 위한 Security-Group Resource 정의
resource "aws_security_group" "Example_sg" {
  name           = "Example_Instance_Connection"
  description    = "Allow SSH Traffic"
  ingress {
    from_port     = 22          # In-Bound 규칙
    to_port       = 22          # 시작 포트번호
    protocol      = "tcp"       # 끝나는 포트번호
    cidr_blocks   = ["0.0.0.0/0"] # 허용 IP 범위
  }
  egress {
    from_port     = 0          # Out-Bound 규칙
    to_port       = 0          # 시작 포트번호
    protocol      = "-1"       # 끝나는 포트번호
    cidr_blocks   = ["0.0.0.0/0"] # 허용 IP 범위
  }
}
```

```
resource "aws_instance" "Example" {
  ami              = "ami-0ea4d4b8dc1e46212"
  instance_type    = "t2.micro"
  vpc_security_group_ids = [aws_security_group.Example_sg.id]
  tags = {
    Name = "Terraform_EC2_Example"
  }
}
```

Iac (Infrastructure as Code) : Terraform

◎ Terraform Resource 참조 (EX.3)

▣ [CMD] : **C:\Terraform>terraform apply**

- ▶ AWS 콘솔 VPC 메뉴에서 Security-Group 생성 및 EC2 Instance와 연결되었는지 확인
- ▶ 생성 확인 후 반드시 테스트에서 사용 한 Resource는 Destroy 명령을 사용하여 Resource 삭제 작업을 수행한다.

[... Plan 수행 ...]

Terraform will perform the following actions:

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: **yes**

aws_instance.Example: Creating...

aws_instance.Example: Still creating... [10s elapsed]

aws_instance.Example: Creation complete after 32s [id=i-076682fc57f273426]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

IaC (Infrastructure as Code) : Terraform

◎ Terraform Resource 참조 (EX.3)

▣ Security-Group 생성 및 EC2 Instance와 연결 확인

인스턴스 (1/1) 정보

인스턴스를 속성 또는 (case-sensitive) 태그로 찾기

인스턴스 상태: running

Name	인스턴스 ID	인스턴스 상태	인스턴스 유형	상태 검사	경보 상태	가용 영역	퍼블릭 IPv4 DNS	퍼블릭 IPv4 ...	탄력적 IP
Terraform_EC...	i-076682fc57f273426	실행 중	t2.micro	2/2개 검사 통과...	경보 없음	ap-northeast-2c	ec2-13-124-100-33.ap-...	13.124.100.33	-

인스턴스: i-076682fc57f273426(Terraform_EC2_Example)

▼ 보안 세부 정보

IAM 역할: -

보안 그룹: sg-0c527c5249e1602b4 (Example_Instance_Connection)

소유자 ID: 398289342227

시작 시간: Fri Aug 04 2023 16:25:11 GMT+0900 (한국 표준시)

▼ 인바운드 규칙

이름	보안 그룹 규칙 ID	포트 범위	프로토콜	원본	보안 그룹	설명
-	sgr-0c5652599c25a057a	22	TCP	0.0.0.0/0	Example_Instance_Connection	-

▼ 아웃바운드 규칙

이름	보안 그룹 규칙 ID	포트 범위	프로토콜	대상	보안 그룹	설명
-	sgr-0911555b50b4fc4d2	전체	전체	0.0.0.0/0	Example_Instance_Connection	-

IaC (Infrastructure as Code) : Terraform

◎ Terraform Data Source

- ▶ Data Source : Data Source는 외부 공급자(AWS)에서 가져온 읽기 전용 정보를 의미한다.
- ▶ Data Source는 새롭게 정보가 생성 되는 것이 아닌, 기존 데이터 정보만 가져와 현재 테라폼 코드에 적용 할 때 사용된다.
- ▶ AWS Data Source (EX) : VPC 정보, Subnet 정보, AMI 정보, IAM 자격증명 정보 등

[Data Resource 정의 구문]

```
data "<PROVIDER>_<TYPE>" "<NAME>" {  
  [ CONFIG ... ]  
}
```

- <PROVIDER> : 공급자 이름
- <TYPE> : 공급자의 리소스의 유형 (EC2, VPC 등)
- <NAME> : 다른 테라폼 코드에서 해당 데이터 소스를 참조하기 위한 식별 이름
- [CONFIG] : 특정 데이터 소스를 조회하기위한 설정 값

[다른 테라폼 코드에서 정의 된 Data Resource 참조하는 구문]

```
data.<PROVIDER>_<TYPE>.<NAME>.<ATTRIBUTE>
```

- <PROVIDER> : 공급자 이름
- <TYPE> : 공급자의 리소스의 유형 (EC2, VPC 등)
- <NAME> : 데이터 소스 이름 지정
- <ATTRIBUTE> : 데이터 소스내의 특정하나의 정보 (VPC Data Source 내의 Subnet 정보)

Iac (Infrastructure as Code) : Terraform

◎ Terraform Data Source (EX.4)

- ▶ 기존 EC2 Instance, Security Group Resource 정의영역 **주석 후 작업**
- ▶ VPC Data Source Docs : <https://registry.terraform.io/providers/hashicorp/aws/latest/docs/data-sources/vpc>
- ▶ Subnet Resource Docs : <https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/subnet>

```
# VPC Data Source 정의
data "aws_vpc" "default_vpc" {
  default = true # default VPC로 지정된 VPC 지정
}

# Data Source를 활용한 Resource VPC Subnet 정의
resource "aws_subnet" "default_vpc_subnet" {
  vpc_id      = data.aws_vpc.default_vpc.id # Subnet을 생성 할 VPC ID
  cidr_block  = "172.31.64.0/20" # Subnet CIDR
  tags = {
    Name = "Terraform_Subnet" # Subnet Tag
  }
}
```

Iac (Infrastructure as Code) : Terraform

◎ Terraform Data Source (EX.4)

▣ [CMD] : **C:\Terraform>terraform apply**

- ▶ AWS 콘솔 VPC 메뉴에서 Default VPC에 새로운 Subnet이 생성되었는지 확인
- ▶ 생성 확인 후 반드시 테스트에서 사용 한 Resource는 Destroy 명령을 사용하여 Resource 삭제 작업을 수행한다.

[... Plan 수행 ...]

Terraform will perform the following actions:

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: **yes**

aws_subnet.default_vpc_subnet: Creating...

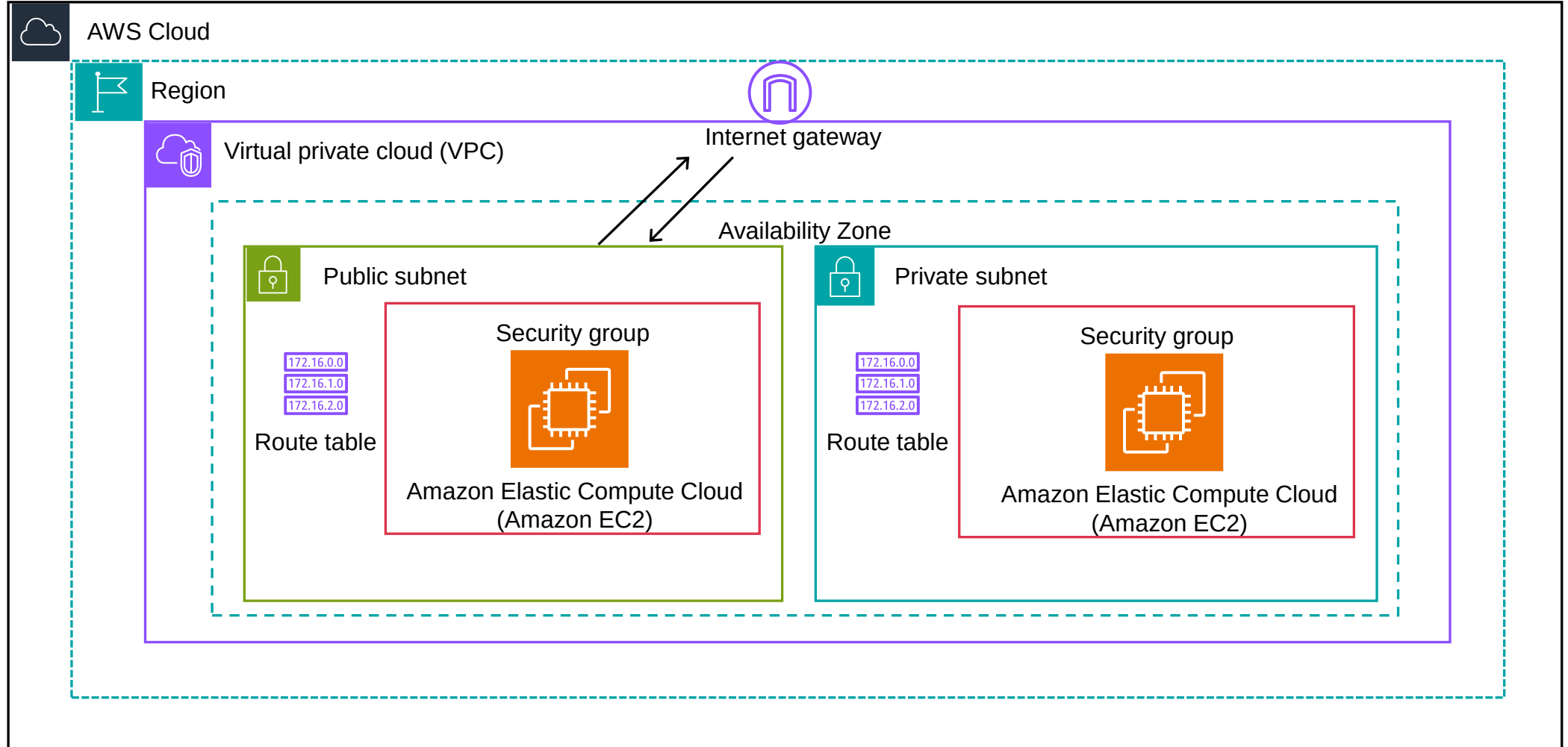
aws_subnet.default_vpc_subnet: Creation complete after 0s [id=subnet-051ada35b3eb3d577]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Iac (Infrastructure as Code) : Terraform

◎ [실습.1] : Terraform Resource & Data source

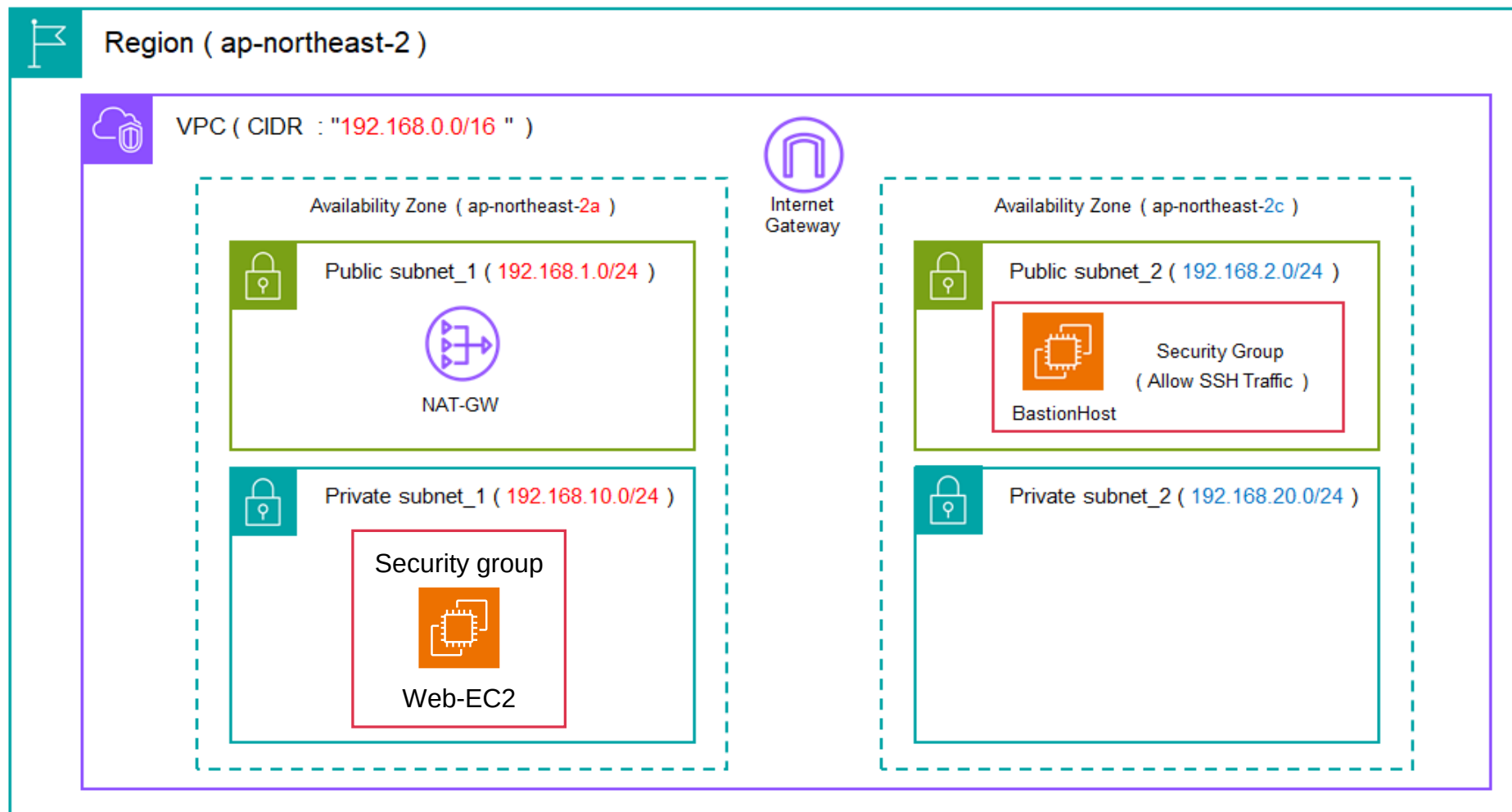
▶ Terraform Code를 작성하여 제시 된 AWS Architecture를 구현 하세요. ([DOCS 문서 활용](#))



Iac (Infrastructure as Code) : Terraform

◎ [실습.2] : Terraform Resource & Data source

▶ Terraform Code를 작성하여 제시 된 AWS Architecture를 구현 하세요. ([DOCS 문서 활용](#))





Variable_Input & Output

Iac (Infrastructure as Code) : Terraform

◎ Terraform Variable_Input

- ▶ Input Variable (입력 변수) : Resource 정의 시 Config 영역에서 사용 할 값을 저장한다.
- ▶ 입력 변수를 활용하여 테라폼 코드의 변경 없이 다양한 Arguments 값을 지정하여 코드의 유연성을 확보 할 수 있다.
- ▶ 입력 변수는 Terraform Module 구조(★★★)를 구현하는데 필수요소가 된다.

```
variable "<NAME>" {  
  description = "설명"  
  default = "기본값 정의"  
  type = "변수 유형"  
}
```

- "NAME" : 입력 변수 이름
- "DESCRIPTION" : 변수 설명 **plan, apply** 명령어를 실행 할 때 함께 출력
- "DEFAULT" : 정의 된 변수에 값이 지정되지 않았을 경우 사용 할 기본 값
- "TYPE" : 테라폼 사용자가 해당 변수에게 전달 할 인자값의 유형을 정의 (제약조건)
- TYPE 종류 : **any, string, number, bool, list, map, set, object, tuple**

Iac (Infrastructure as Code) : Terraform

© Terraform Variable_Input

▶ Terraform 입력 변수는 다양한 형태의 Type을 정의 할 수 있고, 필요에 따라 검증규칙 (Validation) 설정도 가능하다.

```
variable "ex1" {
  description = "Number Variable"
  type        = number
  default     = 1000
}

variable "ex2" {
  description = "List Variable"
  type        = list(any)
  default     = ["A", "B", "C"]
}

variable "ex3" {
  description = "List + Number Variable"
  type        = list(number)
  // List Type + Number Type
  default     = [10, 20, 30]
}
```

```
variable "ex4" {
  description = "Map Variable"
  type        = map(any)
  default = {
    key1 = val1
    key2 = val2
  }
}

variable "ex5" {
  description = "Object Variable"
  type = object({
    name = string
    age  = number
    flag = bool
  })
  default = {
    name = "TEST"
    age  = 20
    flag = true
  }
}
```

Iac (Infrastructure as Code) : Terraform

◎ Terraform Variable_Input

- ▶ 테라폼 모듈구조가 아닌 환경에서 입력변수에 값을 지정하는 방법 3가지 (명령줄 옵션 / 환경변수 등록 / 대화식 처리)
- ▶ 테라폼 모듈구조에서는 테라폼 코드내에서 입력변수의 값을 지정하여 사용하게 된다.

[정의 된 테라폼 변수에 값 저장]

1. 테라폼 명령줄 옵션 "-var" 사용

```
$ terraform (apply or plan) -var "ex1=100"
```

2. 환경변수 사용

```
$ export TF_VAR_ex1=100
$ terraform plan
```

3. 대화식 변수처리

```
$ terraform apply
var.ex1           // 변수 이름
Number Variable   // 변수 description 출력
Enter a value : 100 // 변수에 정의 될 값을 입력받기 위한 대기상태
```

Iac (Infrastructure as Code) : Terraform

◎ Terraform Variable 참조

- ▶ Variable 참조: 입력변수로 정의 된 다양한 Arguments 값을 참조하는 구문
- ▶ 미리 정의되어있는 입력변수를 Resource 정의 시 연결하여 사용한다.

```
[ 테라폼 변수 참조 구문 ( Variable Reference ) ]  
>. var."<VARIABLE_NAME>"
```

```
[ 테라폼 변수 참조 Example ]
```

```
variable "ssh_port" {  
  description = "The Port the Server Will use for SSH Service"  
  type        = number  
}
```

```
resource "aws_security_group" "SG_1" {  
  name = "terraform-instance"  
  ingress {  
    from_port = var.ssh_port  
    to_port   = var.ssh_port  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

Iac (Infrastructure as Code) : Terraform

◎ Terraform Output

- ▶ Output (출력) : 테라폼 코드를 이용하여 Resource가 정의되고, 정의 된 Resource 정보를 출력 할 때 사용된다.
- ▶ 출력 변수를 활용하여 정의 된 Resource의 정보를 AWS 콘솔이 아닌, 명령줄 터미널에서 바로 확인이 가능하다.

[출력(OUTPUT) 정의 구문]

```
output "<NAME>" {  
  value = <VALUE>  
  description = "설명"  
  sensitive = <Bool>  
}
```

- "NAME" : 출력 변수 이름
- "DESCRIPTION" : 출력 변수에 대한 설명
- "SENSITIVE" : 출력 변수에 값을 화면에 표시하지 않음 <"true"> (개인키 or 비밀번호)
- 출력변수 사용예시 : EC2 인스턴스의 Public IP주소를 apply 명령 실행 후 화면에 출력

IaC (Infrastructure as Code) : Terraform

◎ Terraform Output

[테라폼 출력 변수 Example]

```
output "EC2_Pub_IP" {  
  value = aws_instance.ExampleEC2.public_ip  
  description = "EC2 Instance Public IP Address"  
}
```

```
$ terraform apply
```

```
aws_instance.ExampleEC2: Still creating... [20s elapsed]
```

```
aws_instance.ExampleEC2: Creation complete after 22s [id=i-038a3910b33acb968]
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
public_ip = 54.123.45.6
```

```
$ terraform output
```

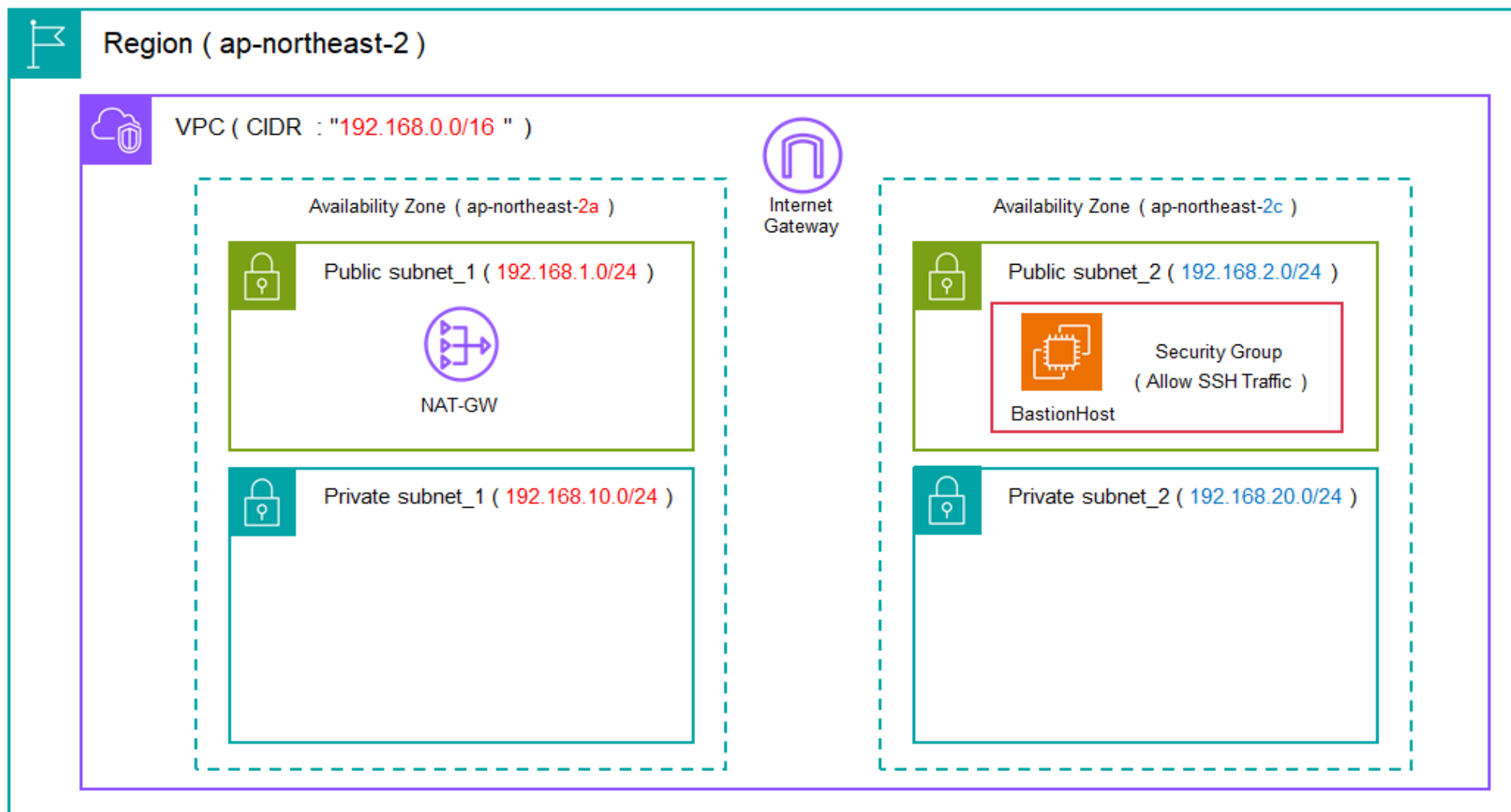
```
public_ip = 54.123.45.6
```

```
$ terraform output public_ip
```

```
54.123.45.6
```

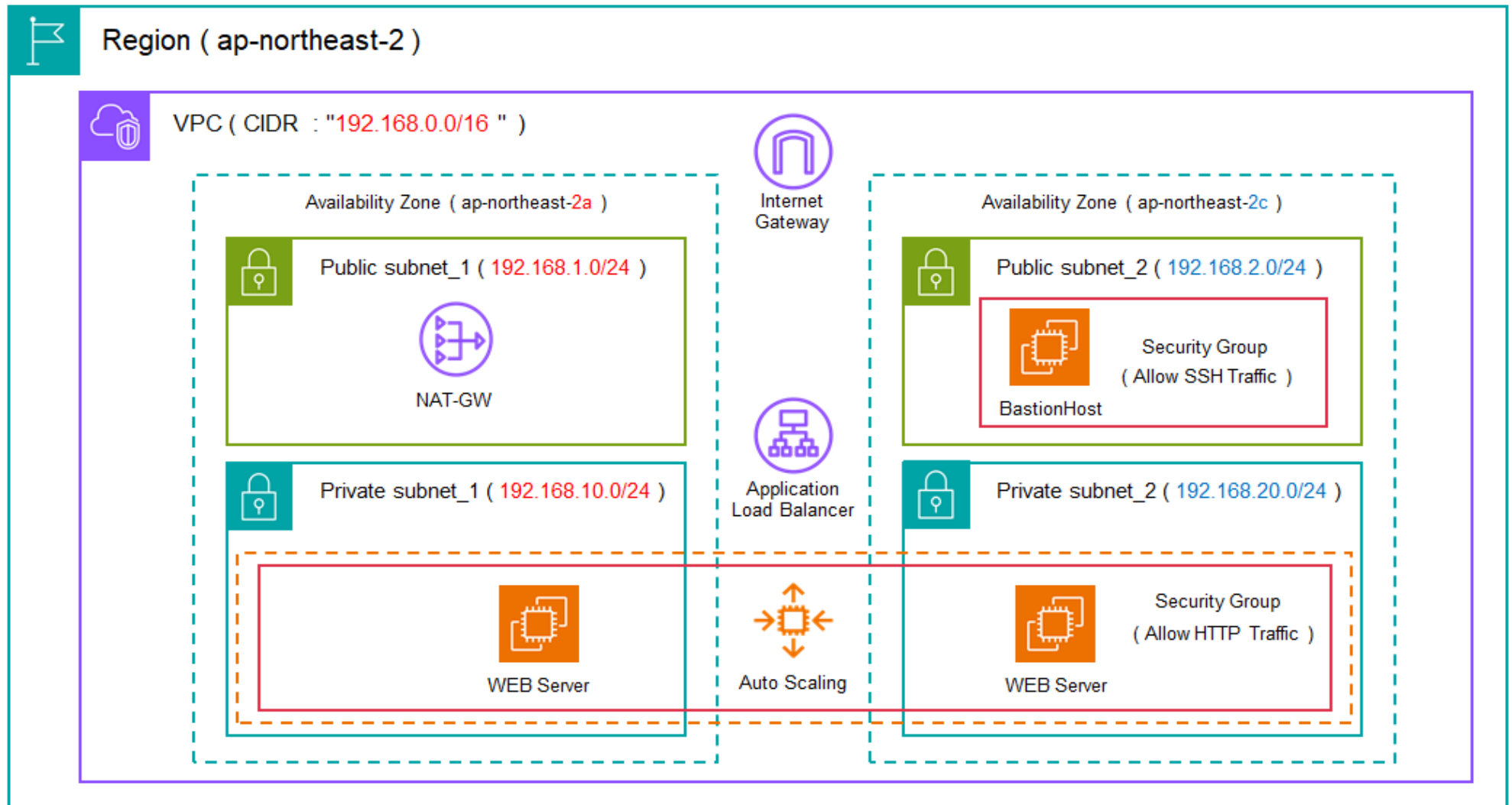
Iac (Infrastructure as Code) : Terraform

© [EX.1] : Terraform Variable_Input & Output (VPC)



Iac (Infrastructure as Code) : Terraform

© [EX.2] : Terraform Variable_Input & Output (ALB & ASG)





Terraform 반복문 & 조건문

IaC (Infrastructure as Code) : Terraform

◎ Terraform 반복문 count (EX.1 ~ EX.2)

- ▶ Count : 유사한 형태의 여러 Resource를 반복하여 정의할때 사용 (EX: AWS IAM USER)
- ▶ Count 특징 : Resource Block 전체를 반복 실행, Resource 내부 인라인 영역을 반복 실행하는 것은 불가능하다.

```
# Python Code
for i in range(0,3,1):
    resource "aws_iam_user" "count_ex1" {
        name = "Terra[i]"
    }

# Terraform Count 활용
resource "aws_iam_user" "count_ex1" {
    count = 3
    name = "Terra.${count.index}"
    # "${...}" = interpolation(보간)
    # 문자열 리터럴 내부에 특정값(변수, 참조)을 추가하기 위해 사용
}
```

범용 프로그래밍 언어의
반복문 형식 (실행 X)

테라폼 count를 활용한
Resource 영역 반복



- AWS IAM User의 이름은 계정내에서 유일해야 한다.
- 서로 다른 IAM User의 이름을 정의하기 위해 반복 Index를 활용한다.
- Terraform Count에서는 반복 Index를 지원한다.

```
# aws_iam_user.count_ex1[0] will be created
+ resource "aws_iam_user" "count_ex1" {
    + arn          = (known after apply)
    + force_destroy = false
    + id           = (known after apply)
    + name         = "Terra.0"
    + path         = "/"
    + tags_all     = (known after apply)
    + unique_id    = (known after apply)
}

# aws_iam_user.count_ex1[1] will be created
+ resource "aws_iam_user" "count_ex1" {
    + arn          = (known after apply)
    + force_destroy = false
    + id           = (known after apply)
    + name         = "Terra.1"
    + path         = "/"
    + tags_all     = (known after apply)
    + unique_id    = (known after apply)
}
```

Iac (Infrastructure as Code) : Terraform

◎ Terraform 반복문 for_each (EX.3 ~ EX.4)

- ▶ 리스트, 집합, 맵을 사용하여 유사한 여러 Resource를 반복하여 정의하거나, Resource 내부 인라인 영역을 반복 정의할때 사용.
- ▶ Count의 제약사항 및 단점을 보완하기위해 for_each 표현식을 사용한다. (count, for_each 는 Module 내부에서는 사용 불가)

☐ Terraform for_each

```
resource "<PROVIDER>_<TYPE>" "<NAME>" {  
  for_each = <COLLECTION>  
  [ CONFIG ... ]  
}
```

- <PROVIDER> : 공급자 이름
- <TYPE> : 공급자의 리소스의 유형 (EC2, VPC 등)
- <NAME> : 다른 테라폼 코드에서 해당 리소스를 참조하기 위한 식별 이름
- <COLLECTION> : 반복 루프를 처리 할 집합(set) 또는 맵(map)
- [CONFIG] : 특정 리소스를 생성하기위한 설정 값 (AMI, INSTANCE 유형 등)

>. Resource 내부에서 for_each 표현식을 사용하는 경우 리스트는 지원되지 않는다.

IaC (Infrastructure as Code) : Terraform

◎ Terraform 조건문 (EX.5 ~ EX.6)

- ▶ Terraform Conditionals 정의 방법 2가지 [Count 매개변수 사용 / for_each & for 표현식 (중첩 For문) 사용]
- ▶ Count 매개변수 : 전체 배포 영역내에서 조건부 Resource 정의를 구현하는 경우 사용된다.

```
variable "env" {  
  type      = string  
  description = "dev or prod ?"  
  default    = "dev"  
}  
  
resource "aws_instance" "Conditionals_ex1" {  
  ami           = "ami-0ea4d4b8dc1e46212"  
  instance_type = "t2.micro"  
  count         = var.env == "dev" ? 1 : 2  
  tags = {  
    Name = "Conditionals_Test"  
  }  
}
```

```
C:\Terraform>terraform plan -var "env=dev"  
# aws_instance.Conditionals_ex1[0] will be created  
+ resource "aws_instance" "Conditionals_ex1"  
Plan: 1 to add, 0 to change, 0 to destroy.
```

```
C:\Terraform>terraform plan -var "env=prod"  
# aws_instance.Conditionals_ex1[0] will be created  
+ resource "aws_instance" "Conditionals_ex1"  
# aws_instance.Conditionals_ex1[1] will be created  
+ resource "aws_instance" "Conditionals_ex1"  
Plan: 2 to add, 0 to change, 0 to destroy.
```



Terraform Module

Iac (Infrastructure as Code) : Terraform

◎ Terraform Module

▶ 테라폼 모듈은 테라폼 코드를 모듈화하여 여러 위치에서 해당 모듈을 사용 할 수 있도록 구현하는 것을 말하며, 테라폼 모듈은 재사용이 가능한 코드형 인프라를 구성하는 가장 핵심적인 요소가 된다.

▶ Terraform Module 사용 예시

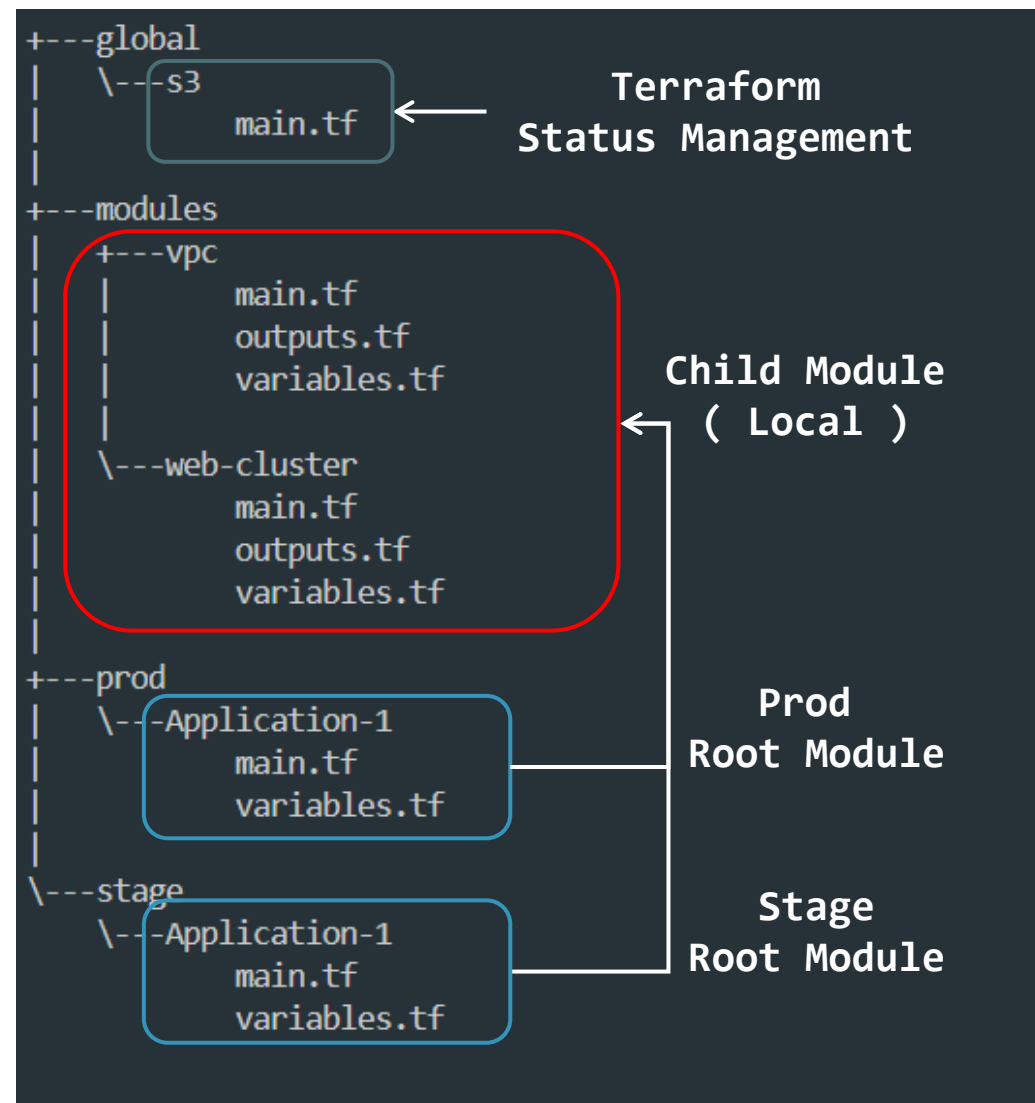
- 회사내부 인프라 환경은 둘 이상의 인프라 환경으로 운영된다.
- 내부 테스트 진행하는 "스테이징" 환경과 실제 클라이언트 액세스가 가능한 "프로덕션" 환경으로 구분.

▶ Terraform Registry (Terraform 외부 Module) [★★★]

- Terraform Registry : Terraform Resource 정의를 쉽게 할 수 있도록 외부에서 제공되는 Terraform Module 저장소를 의미한다.

※ Terraform Module TEST를 위해 VScode 작업 폴더 구조를 이미지와 같이 (Global / Modules / Stage / Prod) 변경한다.
(Key 폴더는 기존 Key 폴더를 그대로 사용)

※ Root Module에서 참조 중인 Module에 변경사항이 발생한 경우 반드시 Terraform Init 명령을 다시 수행해야 한다.



Iac (Infrastructure as Code) : Terraform

© Terraform Module 참조

[Terraform Module 참조 구문]

```
module "<NAME>" {  
  source = "<SOURCE>"  
  [ CONFIG ... ]  
}
```

- <NAME> : 모듈을 식별하기 위한 이름
- <SOURCE> : 모듈 파일의 경로
- [CONFIG] : 해당 모듈을 사용하기 위한 설정 값

```
module "local-vpc" {  
  source = "../../modules/vpc"  
  # Local VPC Module 경로 정의  
  
  name = var.vpc_name  
  cidr = var.vpc_cidr  
  # Local Module에서 지원하는 입력변수 설정  
}
```

▣ Terraform Registry (외부 Module 사용법)

1. Terraform Registry URL: <https://registry.terraform.io/> 접속
 2. Search All Resource : "[AWS](#)" 검색
 3. AWS Provider 선택 ([Official Mark](#))  by:  HashiCorp
 4. terraform-aws-modules / vpc 클릭
 5. Provision Instructions 코드 복사 후 테라폼 코드로 붙여넣기
 6. terraform init 명령을 이용하여 모듈 다운로드 후 사용
- ※ Terraform 외부 모듈을 사용하기 위해서는 반드시 "[Git](#)" 설치
- ※ 외부 모듈 다운로드 == Git Hub Source Code Download

```
module "reg-vpc" {  
  source = "terraform-aws-modules/vpc/aws"  
  version = "4.0.2"  
  # 외부 VPC Module 경로 및 버전 정의  
  
  name = var.vpc_name  
  cidr = var.vpc_cidr  
  # 외부 Module에서 지원하는 입력변수 설정  
}
```

IaC (Infrastructure as Code) : Terraform

© Terraform 원격 백엔드 구성 및 Local Module 정의 (EX.1 ~ Ex.2)

```
terraform {  
  backend "s3" {    상태파일 저장위치 정의  
    key      = "prod/terraform.tfstate"  
  }  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region  = "ap-northeast-2"  
  profile = "terraform_user"  
}
```

```
bucket = "myterraform-bucket-state-choi-t"  
region  = "ap-northeast-2"  
profile = "terraform_user"  
dynamodb_table = "myTerraform-bucket-lock-choi-t"  
encrypt      = true  
              "backend.hcl" 파일 내용  
              ( "backend.hcl" 파일은 변수 사용이 가능 )
```

```
C:\Terraform_module\prod\Application-1>terraform init -backend-config="C:\Terraform_module\global  
\s3\backend.hcl"
```

Initializing the backend...

Backend 구성정보 변경 시
"-backend-config" 옵션 사용

Initializing provider plugins...

- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.12.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

Iac (Infrastructure as Code) : Terraform

◎ Terraform Local Module 활용 (EX.3)

- ▶ Local Module의 경로를 지정하고, Stage 환경에 맞는 입력 변수들의 값을 정의한다.
- ▶ Output 영역은 Module에서 정의하고있는 Output 내용을 참조하여 재정의하여 사용한다.

```
module "vpc" {  
    source      = "../modules/vpc" // Local Module 경로  
    vpc_cidr    = "10.10.0.0/16"    // Module Variable  
    public-1_cidr = "10.10.1.0/24"  
    public-2_cidr = "10.10.2.0/24"  
    private-1_cidr = "10.10.10.0/24"  
    private-2_cidr = "10.10.20.0/24"  
    ssh_port     = 22  
}  
  
output "EC2_Pub_IP" { // Module에서 정의한 Output 재정의  
    value      = module.vpc.EC2_Pub_IP  
    description = "Stage BastionHost Public IP Address"  
}
```

Iac (Infrastructure as Code) : Terraform

◎ Terraform Local Module 활용 (EX.4)

- ▶ 예제.4번 테라폼 코드작성 후 terraform plan 작업 수행 (Web-Cluster Module 구성 시 **Error** 발생)
- ▶ Web-Cluster 내부 Security-Group Resource 정의 영역에서 VPC ID 참조부분에서 Error가 발생하는것을 확인 할 수 있다.
- ▶ 서로 다른 모듈([VPC <-> Web-Cluster](#))로 구성 된 Resource간 참조를 위해서는 "**terraform_remote_state**"를 사용해야한다.
- ▶ "terraform_remote_state"는 Terraform 상태파일에 저장 된 정보를 Data Source로 사용하는것을 말한다.

```
C:\Terraform_module\stage\Application-1>terraform plan
```

```
Error: Reference to undeclared resource
```

```
on ../..modules/web-cluster/main.tf line 6, in resource "aws_security_group" "http_sg":  
6:   vpc id      = aws vpc.my_vpc.id
```

```
A managed resource "aws_vpc" "my_vpc" has not been declared in module.web-cluster.
```

```
Error: Reference to undeclared resource
```

Web-Cluster Module에는
my_vpc Resource가 존재하지 않는다.

```
on ../..modules/web-cluster/main.tf line 130, in resource "aws_lb_target_group" "my_lb_tg":  
130:   vpc id      = aws vpc.my_vpc.id
```

```
A managed resource "aws_vpc" "my_vpc" has not been declared in module.web-cluster.
```

Iac (Infrastructure as Code) : Terraform

◎ [실습.3] : Terraform Module 활용

▶ Terraform Code를 작성하여 문제 조건에 맞는 AWS Resource를 생성하세요.

[요구조건] : 반드시 Stage 환경 Resource 삭제 후 작업

1. Prod 환경 배포작업 수행 (Local Module 활용 [VPC, WEB-Cluster])
2. 상태파일 저장 경로 (Backend Key) : `prod/terraform.tfstate`
3. VPC_CIDR: "`192.168.0.0/16`" (Public, Private "`24 Bit Network`로 적절히 구성")
4. Instance Type: "`m4.large`" / ASG Min: "`2`" / ASG Max: "`4`"
5. Terraform Remote State 활용하여 Module간 참조 구현

※ 모든 테스트 완료 후 반드시 Resource 삭제 작업을 수행

Iac (Infrastructure as Code) : Terraform

© Terraform 외부 Module 활용 VPC (EX.5)

```
# 예제.5 : Terraform 외부 Module 활용 (VPC)
module "stage_vpc" {
  source      = "terraform-aws-modules/vpc/aws" # 외부 VPC Module 정의
  version     = "5.1.1"                         # 외부 VPC Module 버전 정의
  name        = "stage_vpc"                     # VPC 이름
  cidr        = local.stage_cidr                 # CIDR
  azs          = local.azs                       # 가용영역 지정
  public_subnets = local.public_subnets        # Public Subnet 정의
  private_subnets = local.private_subnets      # Private Subnet 정의

  enable_dns_hostnames = "true" # DNS Hostname 사용
  enable_dns_support   = "true" # DNS Support 사용

  enable_nat_gateway = true # NAT Gateway 사용
  single_nat_gateway = true # Single NAT Gateway 설정
  one_nat_gateway_per_az = false # Single NAT Gateway 설정 시 false

  tags = {
    "TerraformManaged" = "true" # 스마트 태그
  }
}
```

Provision Instructions

Copy and paste into your Terraform configuration, insert the variables, and run terraform init:

```
module "vpc" {
  source = "terraform-aws-modules/vpc,"
  version = "5.1.1"
}
```

public_subnets list(string)

Description: A list of public subnets inside the VPC

Default: []

private_subnets list(string)

Description: A list of private subnets inside the VPC

Default: []

Iac (Infrastructure as Code) : Terraform

© Terraform 외부 Module 활용 Security-Group (EX.6)

예제.6 : Terraform 외부 Module 활용 (Security Group)

```
module "SSH_security-group" {
  source      = "terraform-aws-modules/security-group/aws"
  version     = "5.1.0"
  name        = "SSH_SG"           # Security-Group 이름 정의
  description = "SSH Port"        # Security-Group 설명
  vpc_id      = module.stage_vpc.vpc_id # 보안그룹이 생성 될 VPC 정의 (모듈)
  use_name_prefix = "false"        # 이름 뒤 고유번호를 붙이지 않음
```

지역변수를 활용한 InBound 규칙 정의

```
ingress_with_cidr_blocks = [
  {
    from_port = local.ssh_port # 시작 포트번호
    to_port   = local.ssh_port # 끝나는 포트번호
    protocol  = local.tcp_protocol # 프로토콜 타입
    description = "SSH"          # 설명
    cidr_blocks = local.all_network # 허용 IP 범위
  }
]
```

변경 불가능한
지역변수(Locals) 사용



Stage Root Module에서만 사용가능한 지역변수 정의

```
locals {
  stage_cidr    = "10.10.0.0/16"
  azs           = ["ap-northeast-2a", "ap-northeast-2c"]
  public_subnets = ["10.10.1.0/24", "10.10.2.0/24"]
  private_subnets = ["10.10.10.0/24", "10.10.20.0/24"]
  ssh_port      = 22
  http_port     = 80
  https_port    = 443
  any_port      = 0
  any_protocol   = "-1"
  tcp_protocol   = "tcp"
  all_network    = "0.0.0.0/0"
}
```

Provision Instructions

Copy and paste into your Terraform configuration, insert the variables, and run terraform init:

```
module "security-group" {
  source = "terraform-aws-modules/seci
  version = "5.1.0"
}
```

IaC (Infrastructure as Code) : Terraform

© Terraform 외부 Module + Local Module WEB-Cluster (EX.7)

```
module "web-cluster" {  
  source           = "../modules/web-cluster"           외부 Module의 Output을 정의하여  
  instance_type    = "t2.micro"                          Terraform Remote State Data로 활용  
  min_size         = "1"                                  ( 기존 web-cluster Module 수정 )  
  max_size         = "1"  
  vpc_id           = data.terraform_remote_state.remote_data.outputs.vpc_id  
  public_subnet1   = data.terraform_remote_state.remote_data.outputs.public_subnets[0]  
  public_subnet2   = data.terraform_remote_state.remote_data.outputs.public_subnets[1]  
  private_subnet1  = data.terraform_remote_state.remote_data.outputs.private_subnets[0]  
  private_subnet2  = data.terraform_remote_state.remote_data.outputs.private_subnets[1]  
  ssh_sg_id        = data.terraform_remote_state.remote_data.outputs.ssh_sg_id  
  alb_sg_id        = data.terraform_remote_state.remote_data.outputs.alb_sg_id  
  # 외부 모듈을 이용하여 Output 된 Subnet 정보는 List Format이므로, Index를 이용하여 각 Subnet을 선택  
}  
  
output "ALB-DNS" {  
  value          = module.web-cluster.ALB_DNS  
  description    = "Stage Load Balancer Domain Name"  
}
```

Iac (Infrastructure as Code) : Terraform

◎ [실습.4] : Terraform 외부 Module 활용

▶ Terraform Code를 작성하여 문제 조건에 맞는 AWS Resource를 생성하세요.

[요구조건] : 반드시 Stage 환경 Resource 삭제 후 작업

1. Prod 환경 배포작업 수행 (외부 Module [VPC, Security-Group] + Local Module [Web-Cluster])
2. 상태파일 저장 경로 (Backend Key) : `prod/terraform.tfstate`
3. VPC_CIDR: "`192.168.0.0/16`" (Public, Private "`24 Bit Network`로 적절히 구성")
4. Instance Type: "`m4.large`" / ASG Min: "`2`" / ASG Max: "`4`"
5. Terraform Remote State 활용하여 Module간 참조 구현

※ 모든 테스트 완료 후 반드시 Resource 삭제 작업을 수행